

Plane	Representative signals	Question it answers
Accelerator	Memory used, bandwidth utilisation, tokens per second per GPU	Is the hardware earning its cost?
Serving	TTFT, inter-token latency, queue wait, KV cache utilisation, prefix cache hit rate	Is the model server keeping up?
Application	Tokens by tenant and model, cost per request, tool failures, guardrail blocks, eval scores	Are users getting good answers at a sane price?

Here are the signals that predict user pain:

- GPU utilisation percentage is the most misleading number in the stack, because it reports whether any kernel ran rather than whether the device did useful work.
- Queue wait time is the leading indicator, since time to first token rises non-linearly once requests wait for cache capacity.
- End-to-end averages hide streaming stalls, because a response that pauses halfway does complete eventually.
- Goodput, the rate of requests served within SLO, is the figure that tells you when to add capacity.

A server can sit at 95 percent utilisation while producing a fraction of its possible tokens per second, so you must track tokens per second per accelerator alongside it.

How do you trace a request end to end?

One user question can fan out into a retrieval call, three model calls, and two tool invocations. The interesting question is always which of those consumed the time and the tokens.

trace 4f2a... chat.completion	total 8.4s	4,180 tokens \$0.021
└ gateway.guardrail.input	22ms	blocked=false
└ cache.semantic.lookup	41ms	hit=false similarity=0.71
└ retrieval.embed	88ms	model=embed-v3
└ retrieval.search	134ms	k=8 index=support-kb-v11
└ gen_ai.chat attempt 1	3,900ms	in=3,102 out=210 ttft=740ms
└ tool.ticket_lookup	610ms	status=ok scope=read:tickets
└ gen_ai.chat attempt 2	3,510ms	in=3,590 out=278 ttft=690ms
└ gateway.guardrail.output	55ms	redactions=1

Figure 3: A span tree for one request. Token counts and cost belong on spans, not only on dashboards, because attribution is what makes a spike diagnosable

The vendor-neutral way to produce that structure is the OpenTelemetry genAI semantic conventions, which standardise span names, token attributes, and metrics so an LLM call looks the same regardless of which library emitted it. OpenTelemetry graduated from the CNCF in May 2026. Adopt the conventions with your eyes open:

- They remain in development status as of mid-2026 and moved to a dedicated repository in the v1.42.0 release of June 2026, which changed release cadence rather than granting stability.
- Pin the convention version and the instrumentation packages, then inspect a stored trace rather than trusting a documentation example.
- Leave prompt and completion content uncaptured by default, store it separately with its own retention and access controls, and sample rather than capturing everything.

Quality needs its own signals. Automated evaluators score a sample of live traffic, and implicit signals, such as retries, edits, and escalations to a human, cost nothing to collect. Using a model as judge means paying for a second inference and inheriting its variance, so sample rather than judging everything.

The most useful first step to secure an LLM application is drawing the trust boundaries, because the intuitive drawing is wrong (Figure 4).

Prompt injection has no single fix

Prompt injection has held the top position in the OWASP Top 10 for LLM Applications since the list began, and the 2025 edition keeps it at LLM01. The reason is structural: models receive instructions and data through one channel with no reliable separation.

Direct injection is the familiar version. Indirect injection is the dangerous one, where an attacker plants instructions inside a document or a support ticket, and they execute when retrieval pulls that content into context. Since no filter catches everything, design so that a successful injection cannot accomplish much:

- Scope every tool credential to least privilege and authorise downstream calls as the end user rather than as a shared agent identity.
- Validate output against a schema before anything acts on it, and never pass model output into an interpreter, a shell, or a query untreated.
- Require human approval for actions that move money, send external communication, or delete data.
- Restrict outbound network access from tool sandboxes to an allowlist, which turns many exfiltration attempts into blocked connections.
- Cap loop depth, tool calls, and output tokens per session, because unbounded consumption is both denial of service and denial of wallet.